Guia de instrução rápida

❶

CONJUNTO DE INSTRUÇÕES BÁSICO

NOME	MNEMÔNICO	FORMATO	OPERAÇÃO (EM VERILOG)		OPCODE/ FUNCT (HEXA)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0/20_{hexa}$
Add Immediate	addi	I	R[rt] = R[rs] + ImSinExt	(1)(2)	8_{hexa}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + ImSinExt	(2)	9_{hexa}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0/21_{hexa}$
And	and	R	R[rd] = R[rs] & R[rt]		$0/24_{hexa}$
And Immediate	andi	I	R[rt] = R[rs] & ImZeroExt	(3)	c_{hexa}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+ EndBranch	(4)	4_{hexa}
Branch On Not Equal	bne	I	If(R[rs]!=R[rt]) PC=PC+4+EndBranch	(4)	5_{hexa}
Jump	j	J	PC=EndJump	(5)	2_{hexa}
Jump And Link	jal	J	R[31]=PC+4;PC=EndJump	(5)	3_{hexa}
Jump Register	jr	R	PC=R[rs]		$0/08_{hexa}$
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs] +ImSinExt](7:0)}	(2)	$0/24_{hexa}$
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+ ImSinExt](15:0)}	(2)	$0/25_{hexa}$
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hexa}
Load Word	lw	I	R[rt] = M[R[rs]+ ImSinExt]	(2)	$0/23_{hexa}$
Nor	nor	R	R[rd] = ~(R[rs]|R[rt])		$0/27_{hexa}$
Or	or	R	R[rd] = R[rs] | R[rt]		$0/25_{hexa}$
Or Immediate	ori	I	R[rt] = R[rs] | ImZeroExt	(3)	d_{hexa}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		$0/2a_{hexa}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < ImSinExt) ? 1 :0	(2)	a_{hexa}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < ImSinExt) ? 1 : 0	(2)(6)	b_{hexa}
Set Less Than Unsigned	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	$0/2b_{hexa}$
Shift Left Logical	sll	R	R[rd] = R[rs] << shamt		$0/00_{hexa}$
Shift Right Logical	srl	R	R[rd] = R[rs] >> shamt		$0/02_{hexa}$
Store Byte	sb	I	M[R[rs]+ ImSinExt](7:0) = R[rt](7:0)	(2)	28_{hexa}
Store Halfword	sh	I	M[R[rs]+ ImSinExt](15:0) = R[rt](15:0)	(2)	29_{hexa}
Store Word	sw	I	M[R[rs]+ ImSinExt] = R[rt]	(2)	$2b_{hexa}$
Subtract	sub	R	R[rd] = R[rs] – R[rt]	(1)	$0/22_{hexa}$
Subtract Unsigned	subu	R	R[rd] = R[rs] – R[rt]		$0/23_{hexa}$

(1) Pode causar exceção de overflow
(2) ImmSinExt = { 16{imediato[15]}, imediato }
(3) ImmZeroExt = { 16{1b'0}, imediato }
(4) EndBranch = { 14{imediato [15]}, imediato, 2'b0 }
(5) JumpAddr = { PC[31:28], address, 2'b0 }
(6) Operandos considerados números sem sinal (vs. compl. 2)

FORMATOS DE INSTRUÇÃO BÁSICOS

R	opcode	rs	rt	Rd	shamt	funct
	31	26 25	21 20	16 15	11 10	6 5 0
I	opcode	rs	rt	imediato		
	31	26 25	21 20	16 15		0
J	opcode	Endereço				
	31	26 25				0

❷

CONJUNTO DE INSTRUÇÕES BÁSICAS ARITMÉTICAS

NOME	MNEMÔNICO	FORMATO	OPERAÇÃO	OPCODE/FMT/ FT/FUNCT (HEXA)
Branch On FP True	bclt	FI	if(FPcond)PC=PC+4+EndBranch (4)	11/8/1/–
Branch On FP False	bclf	FI	if(!FPcond)PC=PC+4+EndBranch (4)	11/8/0/–
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	0/–/–/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt] (6)	0/–/–/1b
FP Add Single	add.s	FR	F[fd]= F[fs] + F[ft]	11/10/–/0
FP Add Double	Add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}	11/11/–/0
FP Compare Single	C.x.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0	11/10/–/y
FP Compare Double	C.x.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]})? 1 : 0	11/11/–/y
	* (*x* é eq, lt ou le) (*op* é ==, < ou <=) (*y* é 32, 3c ou 3e)			
FP Divide Single	div.s	FR	F[fd] = F[fs]/F[ft]	11/10/–/3
FP Divide Double	div.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]}/ {F[ft],F[ft+1]}	11/11/–/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]	11/10/–/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]}* {F[ft],F[ft+1]}	11/11/–/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs]-F[ft]	11/10/–/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]}-{F[ft],F[ft+1]}	11/11/–/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+ ImSinExt] (2)	31/–/–/–
Load FP Double	ldc1	I	F[rt]=M[R[rs]+ ImSinExt]; F[rt+I]=M[R[rs]+ ImSinExt+4] (2)	35/–/–/–
Move From Hi	mfhi	R	R[rd] = Hi	0/–/–/10
Move From Lo	mflo	R	R[rd] = Lo	0/–/–/12
Move From Control	mfc0	R	R[rd] = CR[rs]	16/0/–/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]	0/–/–/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt] (6)	0/–/–/19
Store FP Single	swc1	I	M[R[rs]+ ImSinExt] = F[rt] (2)	39/–/–/–
Store FP Double	sdc1	I	M[R[rs]+ ImSinExt] = F[rt]; (2) M[R[rs]+ ImSinExt+4] = F[rt+1]	3d/–/–/–

FORMATOS DAS INSTRUÇÕES DE PONTO FLUTUANTE

FR	opcode	fmt	ft	fs	fd	funct
	31	26 25	21 20	16 15	11 10	6 5 0
FI	opcode	fmt	ft	imediato		
	31	26 25	21 20	16 15		0

CONJUNTO DE PSEUDO-INSTRUÇÕES

NOME	MNEMÔNICO	OPERAÇÃO
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	Ble	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	Bge	if(R[rs]>=R[rt]) PC = Label
Load Immediate	Li	R[rd] = imediato
Move	Move	R[rd] = R[rs]

NOME DO REGISTRADOR, NÚMERO, USO E CONVENÇÃO DE CHAMADA

NOME	NÚMERO	USO	PRESERVADO ENTRE CHAMADAS?
$zero	0	O valor constante 0	N.A.
$at	1	Temporário do montador	Não
$v0-$v1	2-3	Valores para resultados de função e avaliação de expressão	Não
$a0-$a3	4-7	Argumentos	Não
St0-$t7	8-15	Temporários	Não
$s0-$s7	16-23	Temporários salvos	Sim
$t8-$t9	24-25	Temporários	Não
$k0-$k1	26-27	Reservado para kernel do SO	Não
$gp	28	Ponteiro global	Sim
$sp	29	Stack Pointer	Sim
$fp	30	Frame Pointer	Sim
$ra	31	Endereço de retorno	Sim

Copyright 2005 by Elsevier, Inc. Todos os direitos reservados. De Patterson e Hennessy, *Organização e Projeto de Computador*, 3ª ed.

Linguagem de máquina do MIPS

Nome	Formato	Exemplo						Comentários
add	R	0	18	19	17	0	32	add $s1,$s2,$s3
sub	R	0	18	19	17	0	34	sub $s1,$s2,$s3
lw	I	35	18	17	100			lw $s1,100($s2)
sw	I	43	18	17	100			sw $s1,100($s2)
and	R	0	18	19	17	0	36	and $s1,$s2,$s3
or	R	0	18	19	17	0	37	or $s1,$s2,$s3
nor	R	0	18	19	17	0	39	nor $s1,$s2,$s3
andi	I	12	18	17	100			andi $s1,$s2,100
ori	I	13	18	17	100			ori $s1,$s2,100
sll	R	0	0	18	17	10	0	sll $s1,$s2,10
srl	R	0	0	18	17	10	2	srl $s1,$s2,10
beq	I	4	17	18	25			beq $s1,$s2,100
bne	I	5	17	18	25			bne $s1.$s2,100
slt	R	0	18	19	17	0	42	slt $s1,$s2,$s3
j	J	2	2500					j 10000 (ver Seção 2.9)
Tamanho do campo		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Todas as instruções MIPS de 32 bits
Formato R	R	op	rs	rt	rd	shamt	funct	Formato das instruções aritméticas
Formato I	I	op	rs	rt	endereço			Formato para transferências de dados e desvios

FIGURA 2.13 A linguagem de máquina do MIPS revelada até a Seção 2.6. As partes destacadas mostram as estruturas do MIPS introduzidas na Seção 2.6. O formato J, usado para instruções de jump, é explicado na Seção 2.9, que também explica os valores apropriados para os campos de endereço das instruções de desvio.

Verifique você mesmo

A linguagem C possui muitas instruções para decisões e loops, enquanto o MIPS possui poucas. Quais dos seguintes itens explicam ou não explicam esse desequilíbrio? Por quê?

1. Mais instruções de decisão tornam o código mais fácil de ler e entender.
2. Menos instruções de decisão simplificam a tarefa da camada inferior responsável pela execução.
3. Mais instruções de decisão significam menos linhas de código, o que geralmente reduz o tempo de codificação.
4. Mais instruções de decisão significam menos linhas de código, o que geralmente resulta na execução de menos operações.

Por que a linguagem C oferece dois conjuntos de operadores para AND (& e &&) e dois conjuntos de operadores para OR (| e ||), enquanto o MIPS não faz isso?

1. As operações lógicas AND e OR implementam & e |, enquanto os desvios condicionais implementam && e ||.
2. A afirmativa anterior é o contrário: && e || correspondem a operações lógicas, enquanto & e | são mapeados para desvios condicionais.
3. Elas são redundantes e significam a mesma coisa: && e || são simplesmente herdados da linguagem de programação B, a antecessora do C.

Nome	Número do registrador	Uso	Preservado na chamada?
$zero	0	o valor constante 0	n.a.
$v0-$v1	2-3	valores para resultados e avaliação de expressões	não
$a0-$a3	4-7	Argumentos	não
$t0-$t7	8-15	Temporários	não
$s0-$s7	16-23	Valores salvos	sim
$t8-$t9	24-25	Mais temporários	não
$gp	28	Ponteiro global	sim
$sp	29	stack pointer	sim
$fp	30	frame pointer	sim
$ra	31	Endereço de retorno	sim

FIGURA 2.18 Convenções de registradores MIPS. O registrador 1, chamado `$at`, é reservado para o montador (ver Seção 2.10), e os registradores 26-27, chamados `$k0`–`$k1`, são reservados para o sistema operacional.

Operandos MIPS

Nome	Exemplo	Comentários
32 registradores	$s0-$s7, $t0-$t9, $zero, $a0-$a3, $v0-$v1, $gp, $fp, $sp, $ra	Locais rápidos para dados. No MIPS, os dados precisam estar em registradores para a realização de operações aritméticas. O registrador MIPS $zero sempre é igual a 0. $gp (28) é o ponteiro global. $sp (29) é o stack pointer. $fp é o frame pointer, e $ra (31) é o endereço de retorno.
2^{30} words na memória	Memória[0], Memória[4]..... Memória[4294967292]	Acessadas apenas por instruções de transferência de dados no MIPS. O MIPS utiliza endereços em bytes, de modo que os endereços em words seqüenciais diferem em 4 vezes. A memória contém estruturas de dados, arrays e spilled registers, como aqueles salvos nas chamadas de procedimento.

Assembly do MIPS

Categoria	Instrução	Exemplo	Significado	Comentários
Aritmética	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	Três operandos; dados nos registradores
	subtract	sub $s1,$s2,$s3	$s1 = $s2- $s3	Três operandos; dados nos registradores
Transferência de dados	load word	lw $s1,100($s2)	$s1 = Memória[$s2 + 100]	Dados da memória para o registrador
	store word	sw $s1,100($s2)	Memória[$s2 + 100] = $s1	Dados do registrador para a memória
Lógica	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	Três operadores em registrador; AND bit a bit
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	Três operadores em registrador; OR bit a bit
	nor	nor $s1,$s2,$s3	$s1 = ~($s2 | $s3)	Três operadores em registrador; NOR bit a bit
	and immediate	andi $s1,$s2,100	$s1 = $s2 & 100	AND bit a bit registrador com constante
	or immediate	ori $s1,$s2,100	$s1 = $s2 | 100	OR bit a bit registrador com constante
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Deslocamento à esquerda por constante
	shift right logical	srl $s1,$s2,10	$s1 = $s2 >> 10	Deslocamento à direita por constante
Desvio condicional	branch on equal	beq $s1,$s2,L	if ($s1 == $s2) go to L	Testa igualdade e desvia
	branch on not equal	bne $s1,$s2,L	if ($s1 != $s2) go to L	Testa desigualdade e desvia
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compara menor que; usado com beq, bne
	set on less than immediate	slt $s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compara menor que imediato; usado com beq, bne
Desvio incondicional	jump	j L	go to L	Desvia para endereço de destino
	jump register	jr $ra	go to $ra	Para retorno de procedimento
	jump and link	jal L	$ra = PC + 4, go to L	Para chamada de procedimento

FIGURA 2.19 Arquitetura MIPS revelada até a Seção 2.7. As partes destacadas mostram as estruturas do assembly do MIPS introduzidas na Seção 2.7. O formato J, usado para instruções de jump e jump-and-link, é explicado na Seção 2.9.